WEEK-7

AIM: Implementation of the following Page Replacement Algorithms a) FIFO b) LRU c) optimal

A) FIFO

ALGORIHTM:

FIFO (First-In, First-Out) replaces the page that has been in memory for the longest time.

1. Start with an empty page frame.
2. Read the pages in the given reference string one by one.
3. If the page is already in memory, it is a hit.
4. If the page is not in memory, it is a page fault.
5. If an empty frame is available, place the page in it.
6. If all frames are occupied, replace the page that entered memory first.
7. Continue until all pages in the reference string are processed.
8. Count the total number of page faults.

PROGRAM:

```
#include <stdio.h>
int main() {
int pages[50], frames[10];
int n, f, i, j, k = 0;
int pageFaults = 0, pageHits = 0;
int found;
printf("Enter the number of pages: ");
scanf("%d", &n);
printf("Enter the page reference string:\n");
for (i = 0; i < n; i++)
scanf("%d", &pages[i]);
printf("Enter the number of frames: ");
scanf("%d", &f);
for (i = 0; i < f; i++)
frames[i] = -1;
printf("\nPage\tFrames\t\tStatus\n");
for (i = 0; i < n; i++) {
found = 0;
// Check if page is already present
for (j = 0; j < f; j++) {
if (frames[j] == pages[i]) {
        found = 1;
        break;   }
```

```
}
if (found) {
pageHits++;
printf("%d\t", pages[i]);
for (j = 0; j < f; j++) {
        if (frames[j] != -1)
                printf("%d ", frames[j]);
        else
             printf("- ");
}
printf("\tHit\n");
}
else {
// Replace page using FIFO
frames[k] = pages[i];
k = (k + 1) % f;
pageFaults++;
printf("%d\t", pages[i]);
for (j = 0; j < f; j++) {
        if (frames[j] != -1)
                printf("%d ", frames[j]);
        else
                printf("- ");
}
printf("\tPage Fault\n");
}
}
printf("\nTotal Page Faults = %d\n", pageFaults);
printf("Total Page Hits   = %d\n", pageHits);
return 0;
}
```

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

OUTPUT:

TEST CASE 1:

```
\25071A6236\FIFO(pag
Enter the number of pages: 20
Enter the page reference string:
7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1
Enter the number of frames: 3

Page    Frames          Status
7       7 - -   Page Fault
0       7 0 -   Page Fault
1       7 0 1   Page Fault
2       2 0 1   Page Fault
0       2 0 1   Hit
3       2 3 1   Page Fault
0       2 3 0   Page Fault
4       4 3 0   Page Fault
2       4 2 0   Page Fault
3       4 2 3   Page Fault
0       0 2 3   Page Fault
3       0 2 3   Hit
2       0 2 3   Hit
1       0 1 3   Page Fault
2       0 1 2   Page Fault
0       0 1 2   Hit
1       0 1 2   Hit
7       7 1 2   Page Fault
0       7 0 2   Page Fault
1       7 0 1   Page Fault

Total Page Faults = 15
Total Page Hits   = 5
```

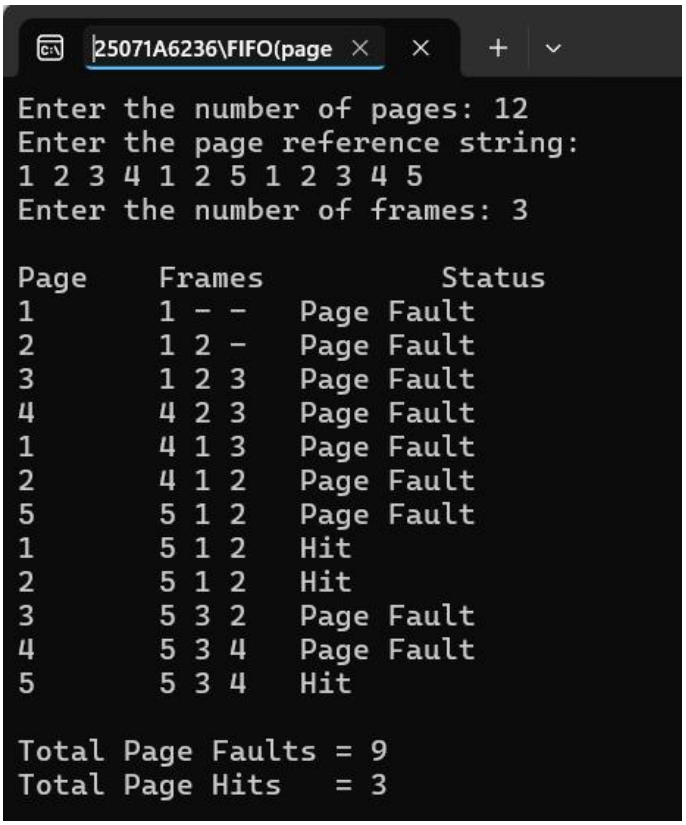
TEST CASE 2:

```
\25071A6236\FIFO(pag
Enter the number of pages: 12
Enter the page reference string:
0 2 1 6 4 0 1 0 3 1 2 1
Enter the number of frames: 3

Page    Frames          Status
0       0 - -   Page Fault
2       0 2 -   Page Fault
1       0 2 1   Page Fault
6       6 2 1   Page Fault
4       6 4 1   Page Fault
0       6 4 0   Page Fault
1       1 4 0   Page Fault
0       1 4 0   Hit
3       1 3 0   Page Fault
1       1 3 0   Hit
2       1 3 2   Page Fault
1       1 3 2   Hit

Total Page Faults = 9
Total Page Hits   = 3
```

TEST CASE 3:


25071A6236\FIFO(page
Enter the number of pages: 12
Enter the page reference string:
1 2 3 4 1 2 5 1 2 3 4 5
Enter the number of frames: 3
Page Frames Status
1 1 - - Page Fault
2 1 2 - Page Fault
3 1 2 3 Page Fault
4 4 2 3 Page Fault
1 4 1 3 Page Fault
2 4 1 2 Page Fault
5 5 1 2 Page Fault
1 5 1 2 Hit
2 5 1 2 Hit
3 5 3 2 Page Fault
4 5 3 4 Page Fault
5 5 3 4 Hit
Total Page Faults = 9
Total Page Hits = 3

B) LRU (LEAST RECENTLY USED)

ALGORIHTM:

LRU (Least Recently Used)** replaces the page that has not been used for the longest period of time.

1. **Start**
2. Read the number of pages in the reference string.
3. Read the page reference string.
4. Read the number of available frames.
5. Initialize all frames as empty.
6. For each page in the reference string:
7. Check whether the page is already present in any frame.
8. If the page is present:

\- Mark it as a **Page Hit**. - Update its recent-use information.

- If the page is not present:

\- Mark it as a **Page Fault**. - If an empty frame is available, insert the page into it. - Otherwise, find the page that was **least recently used**. - Replace that page with the new page.

7. Repeat Step 6 until all pages in the reference string are processed.
8. Display the total number of **Page Faults** and **Page Hits**.

9.**Stop**.

PROGRAM:

PROGRAM:

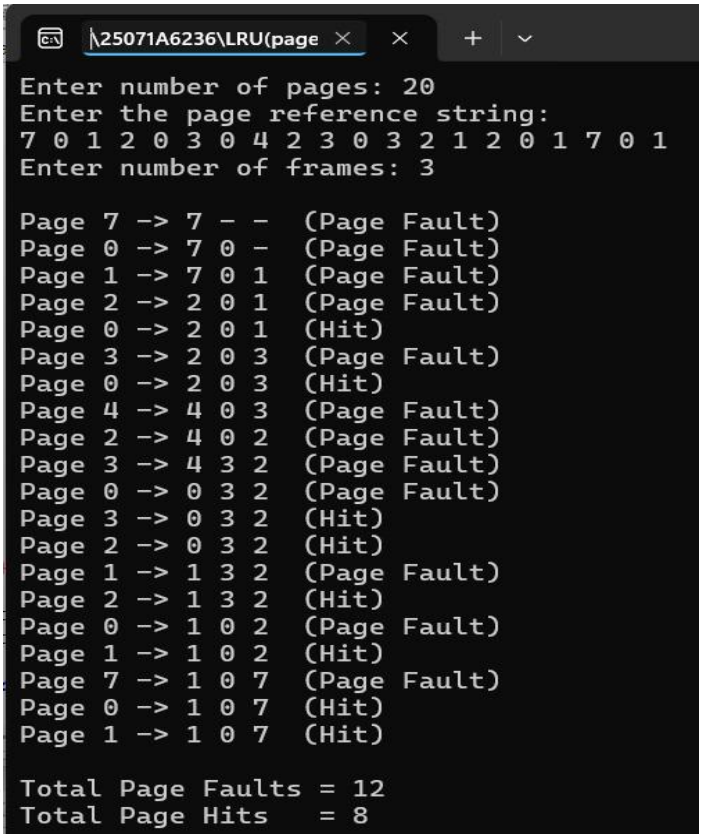
```
#include <stdio.h>
int main() {
int pages[50], frames[10];
int n, f, i, j, k;
int pageFaults = 0, pageHits = 0;
int flag, pos, least;
printf("Enter number of pages: ");
scanf("%d", &n);
printf("Enter the page reference string:\n");
for (i = 0; i < n; i++)
scanf("%d", &pages[i]);
printf("Enter number of frames: ");
scanf("%d", &f);
for (i = 0; i < f; i++)
frames[i] = -1;
for (i = 0; i < n; i++) {
flag = 0;
for (j = 0; j < f; j++)
{     if (frames[j] == pages[i])    {
          flag = 1;
          break;
          }  }
 if (flag == 1)  {
          pageHits++;    }
else  {
          pageFaults++;
          pos = -1;
          for (j = 0; j < f; j++)  {
          if (frames[j] == -1)  {
                    pos = j;
                    break;
          } }
```

```
            if (pos == -1)  {
          int lastUsed[10];

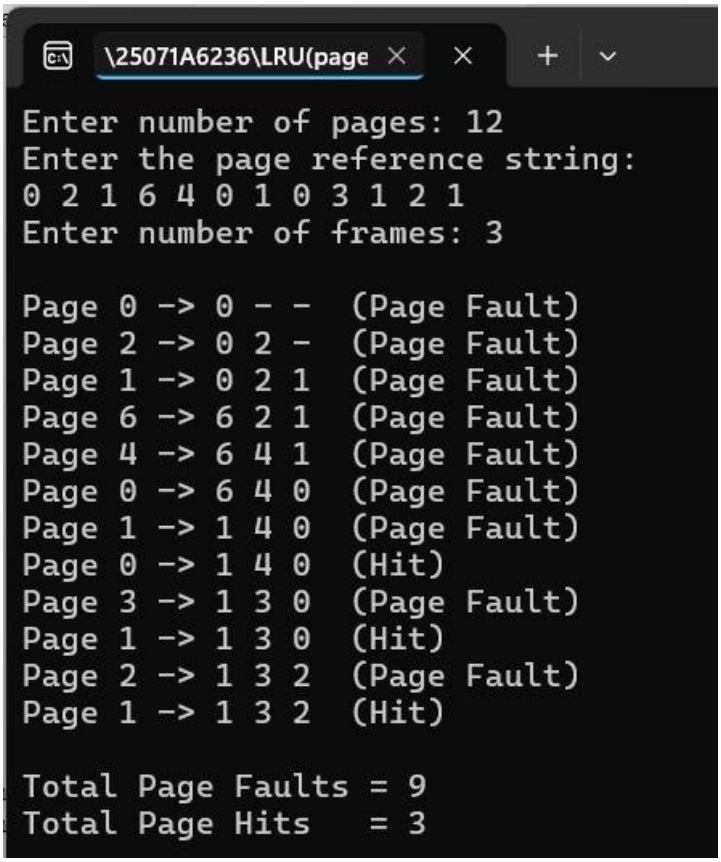
          for (j = 0; j < f; j++)   {
                        lastUsed[j] = -1;
                        for (k = i - 1; k >= 0; k--)   {
                        if (frames[j] == pages[k])
                        {    lastUsed[j] = k;
                        break;  }
                        }  }
        least = lastUsed[0];
        pos = 0;
            for (j = 1; j < f; j++)  {
                        if (lastUsed[j] < least )  {
                        least = lastUsed[j];
                        pos = j;
                        } } }
    frames[pos] = pages[i];    }
    printf("\nPage %d -> ", pages[i]);
    for (j = 0; j < f; j++)   {
            if (frames[j] == -1)
            printf("- ");
            else
            printf("%d ", frames[j]);   }
    if (flag == 1)
            printf(" (Hit)");
  else
          printf(" (Page Fault)");  }
    printf("\n\nTotal Page Faults = %d", pageFaults);
    printf("\nTotal Page Hits   = %d\n", pageHits);
    return 0;    }
```

OUTPUT:

TEST CASE 1:



TEST CASE 2:



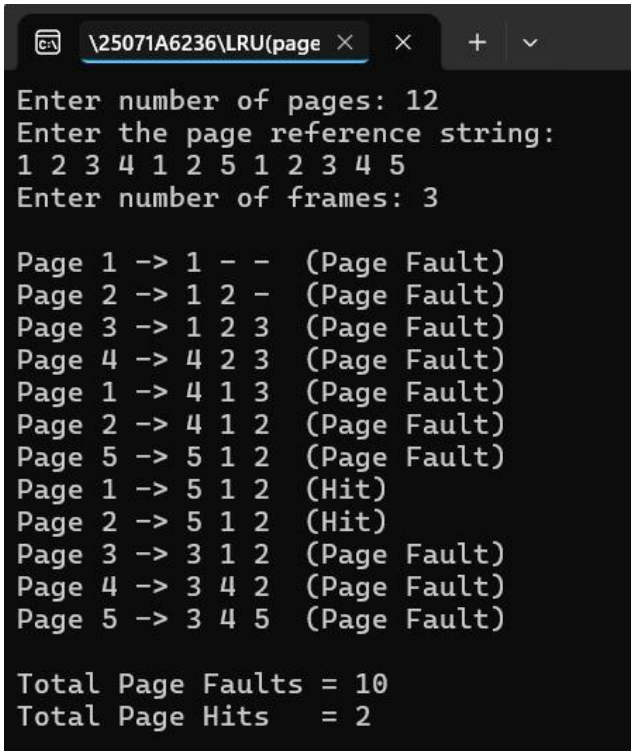
VNR VJIET

Name of the Laboratory:

Name of the Experiment:

Experiment No: **Date:**

TEST CASE 3:



C) OPTIMAL

ALGORITHM:

Optimal Page Replacement replaces the page that will not be used for the longest period of time in the future.

1. **Start**
2. Read the page reference string.
3. Read the number of frames available
4. Read the number of pages in the reference string.
5. Initialize all frames to empty.
6. For each page in the reference string:
7. Check whether the page is already present in any frame.
8. If the page is present, it is a **Page Hit**.
9. If the page is not present, it is a **Page Fault**.
10. If a page fault occurs:
11. If an empty frame is available, insert the page into that frame.
12. Otherwise, check the future references of all pages currently in the frames.
13. Find the page whose next occurrence is **farthest in the future**.
14. If a page will not be used again, select that page for replacement.
15. Replace the selected page with the new page.
16. Repeat Steps 6–7 until all pages in the reference string are processed.
17. Display the total number of **Page Faults** and **Page Hits**.
18. **Stop**.

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

PROGRAM:

```
#include <stdio.h>

    int main()
    {
    int pages[50], frames[10];
    int n, f, i, j, k;
    int pageFaults = 0, pageHits = 0;
    int flag, pos, farthest, future;
    printf("Enter number of pages: ");
    scanf("%d", &n);
    printf("Enter the page reference string:\n");
    for (i = 0; i < n; i++)
    scanf("%d", &pages[i]);
    printf("Enter number of frames: ");
    scanf("%d", &f);
    for (i = 0; i < f; i++)
    frames[i] = -1;
    for (i = 0; i < n; i++)  {
    flag = 0;
    for (j = 0; j < f; j++)   {
        if (frames[j] == pages[i])  {
            flag = 1;
            break;   }  }
    if (flag == 1)  {
            pageHits++;
    }
    else
    {
            pageFaults++;
            pos = -1;
            for (j = 0; j < f; j++)
            {
            if (frames[j] == -1)
            {
                    pos = j;
                    break;   } }
            if (pos == -1)
            {
            farthest = -1;
            pos = -1;
            for (j = 0; j < f; j++)  {
                    future = -1;
            for (k = i + 1; k < n; k++)   {
```

```
                    if (frames[j] == pages[k])
                    {
                    future = k;
                    break;
                    }  }
                    if (future == -1)
                    {
                    pos = j;
                    break;
                    }
                    if (future > farthest)
                    {
                    farthest = future;
                    pos = j;    }}

          frames[pos] = pages[i];    }

printf("\nPage %d -> ", pages[i]);
for (j = 0; j < f; j++)
{
          if (frames[j] == -1)
          printf("- ");
          else
          printf("%d ", frames[j]);
}

if (flag == 1)
          printf("(Hit)");
else
          printf("(Page Fault)");    }

printf("\n\nTotal Page Faults = %d", pageFaults);
printf("\nTotal Page Hits   = %d\n", pageHits);
return 0;
   }
```

VNR VJIET

Name of the Laboratory:

Name of the Experiment:

Experiment No: **Date:**

OUTPUT:

TEST CASE 1:

```
25071A6236\OPTIMAL(
Enter number of pages: 20
Enter the page reference string:
7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1
Enter number of frames: 3

Page 7 -> 7 - - (Page Fault)
Page 0 -> 7 0 - (Page Fault)
Page 1 -> 7 0 1 (Page Fault)
Page 2 -> 2 0 1 (Page Fault)
Page 0 -> 2 0 1 (Hit)
Page 3 -> 2 0 3 (Page Fault)
Page 0 -> 2 0 3 (Hit)
Page 4 -> 2 4 3 (Page Fault)
Page 2 -> 2 4 3 (Hit)
Page 3 -> 2 4 3 (Hit)
Page 0 -> 2 0 3 (Page Fault)
Page 3 -> 2 0 3 (Hit)
Page 2 -> 2 0 3 (Hit)
Page 1 -> 2 0 1 (Page Fault)
Page 2 -> 2 0 1 (Hit)
Page 0 -> 2 0 1 (Hit)
Page 1 -> 2 0 1 (Hit)
Page 7 -> 7 0 1 (Page Fault)
Page 0 -> 7 0 1 (Hit)
Page 1 -> 7 0 1 (Hit)

Total Page Faults = 9
Total Page Hits   = 11
```

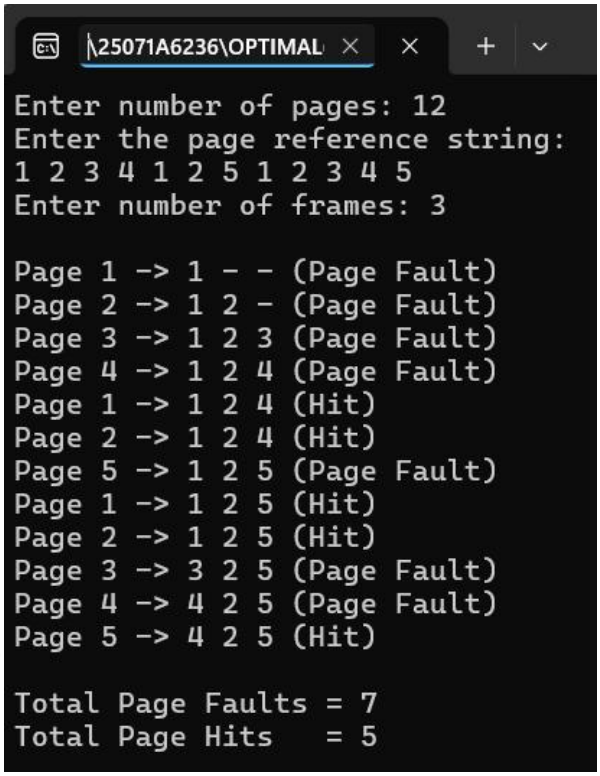
TEST CASE 2:

```
\25071A6236\OPTIMAL
Enter number of pages: 12
Enter the page reference string:
0 2 1 6 4 0 1 0 3 1 2 1
Enter number of frames: 3

Page 0 -> 0 - - (Page Fault)
Page 2 -> 0 2 - (Page Fault)
Page 1 -> 0 2 1 (Page Fault)
Page 6 -> 0 6 1 (Page Fault)
Page 4 -> 0 4 1 (Page Fault)
Page 0 -> 0 4 1 (Hit)
Page 1 -> 0 4 1 (Hit)
Page 0 -> 0 4 1 (Hit)
Page 3 -> 3 4 1 (Page Fault)
Page 1 -> 3 4 1 (Hit)
Page 2 -> 2 4 1 (Page Fault)
Page 1 -> 2 4 1 (Hit)

Total Page Faults = 7
Total Page Hits   = 5
```

TEST CASE 3:


\25071A6236\OPTIMAL
Enter number of pages: 12
Enter the page reference string:
1 2 3 4 1 2 5 1 2 3 4 5
Enter number of frames: 3
Page 1 -> 1 - - (Page Fault)
Page 2 -> 1 2 - (Page Fault)
Page 3 -> 1 2 3 (Page Fault)
Page 4 -> 1 2 4 (Page Fault)
Page 1 -> 1 2 4 (Hit)
Page 2 -> 1 2 4 (Hit)
Page 5 -> 1 2 5 (Page Fault)
Page 1 -> 1 2 5 (Hit)
Page 2 -> 1 2 5 (Hit)
Page 3 -> 3 2 5 (Page Fault)
Page 4 -> 4 2 5 (Page Fault)
Page 5 -> 4 2 5 (Hit)
Total Page Faults = 7
Total Page Hits = 5

VNR VJIET

Name of the Laboratory:

_____

Name of the Experiment:_____

Experiment No:_____ **Date:**_____